

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: COMPUTER ARCHITECTURE

\

COURSE CODE: CSA12

COURSE OUTCOME COVERED

CO1: Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking (BL4) Assessment Tool Weightage: 40%

QUESTIONS:5

Total Marks:25

ANNEXURE A

APPLICATION BASED QUESTIONS

Code of Conduct:

I Thirumalapudi Pranavi (192411222) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

T. Pranavi

Signature of the student:

1. A smart city surveillance system processes continuous video feeds from multiple cameras, where large volumes of data are transferred between I/O devices, memory, and CPU. During peak hours, the system experiences lag and delayed response in threat detection. Analyze how the interaction between CPU, memory, and I/O contributes to this issue, and explain how different bus structures (single-bus vs multi-bus) and improvements in instruction execution cycle can enhance system performance.
2. A real-time stock trading application running on a processor executes millions of instructions per second, but users report delays in transaction processing during high load. The system has a high CPI due to frequent memory access and branch instructions. Examine how the fetch–decode–execute cycle impacts execution time in this scenario, and propose methods to reduce CPI and improve overall CPU performance using suitable architectural enhancements.
3. An embedded system based on an 8085 microprocessor is used in an industrial temperature control unit, where sensors provide input and actuators respond accordingly. However, incorrect temperature readings are occasionally processed due to improper use of addressing modes in the program. Discuss how different addressing modes influence data access, identify possible causes of such errors, and suggest how the instruction set of 8085 can be effectively used to ensure accurate and reliable operation.

4. A computing system uses an 8086 microprocessor with segmented memory for executing multiple programs simultaneously, but it encounters issues like incorrect data access and stack overflow errors. Evaluate how segmentation (code, data, and stack segments) works in 8086, analyze how improper segment handling can lead to such problems, and explain how instruction execution and addressing modes must be managed to ensure correct program execution.
5. A data communication system represents packet information in hexadecimal format for ease of debugging and transmission, but a software engineer mistakenly interprets values, leading to incorrect data processing. Analyze the importance of number system conversions between hexadecimal and binary in such systems, explain how errors in conversion can affect instruction execution, and discuss how efficient CPU design and benchmarking can help detect and minimize such issues in real-time systems.

CO1-ASSESSMENT TOOL 1

Question 1: Smart City Surveillance System

Introduction:

A smart city surveillance system is an advanced security solution that continuously monitors public places using multiple high-definition cameras, sensors, and intelligent computing devices. These systems are widely used in airports, railway stations, shopping malls, traffic management, and public safety applications to detect suspicious activities in real time. The surveillance cameras continuously generate large volumes of video data, which are transferred through Input/Output (I/O) devices to the system memory and processed by the Central Processing Unit (CPU). The CPU performs image processing, object detection, facial recognition, and threat analysis before generating alerts for security personnel. During peak hours, hundreds of cameras transmit video simultaneously, creating heavy traffic between I/O devices, memory, and the CPU. As a result, the processor spends more time waiting for data transfers instead of executing instructions, leading to system lag and delayed threat detection. The efficiency of such systems depends on proper coordination between the CPU, memory, and I/O devices, as well as the use of an efficient bus architecture and optimized instruction execution cycle. Understanding these computer organization concepts helps engineers design faster, more reliable, and scalable surveillance systems capable of handling real-time data processing efficiently.

Theory:

Computer Organization deals with the internal structure and operation of a computer system, focusing on how the CPU, memory, and I/O devices communicate with each other. The CPU acts as the brain of the system and executes instructions using the Fetch–Decode–Execute cycle. During the Fetch stage, instructions and data are retrieved from memory. In the Decode stage, the control unit interprets the instruction, and during the Execute stage, the Arithmetic Logic Unit (ALU) performs the required operation. The results are then stored back in memory.

The communication between these components takes place through a system bus, which consists of three major buses:

- Data Bus – Transfers data between CPU, memory, and I/O devices.

- Address Bus – Carries memory addresses to identify data locations.
- Control Bus – Carries control signals such as read, write, interrupt, and clock signals.

There are two common bus architectures:

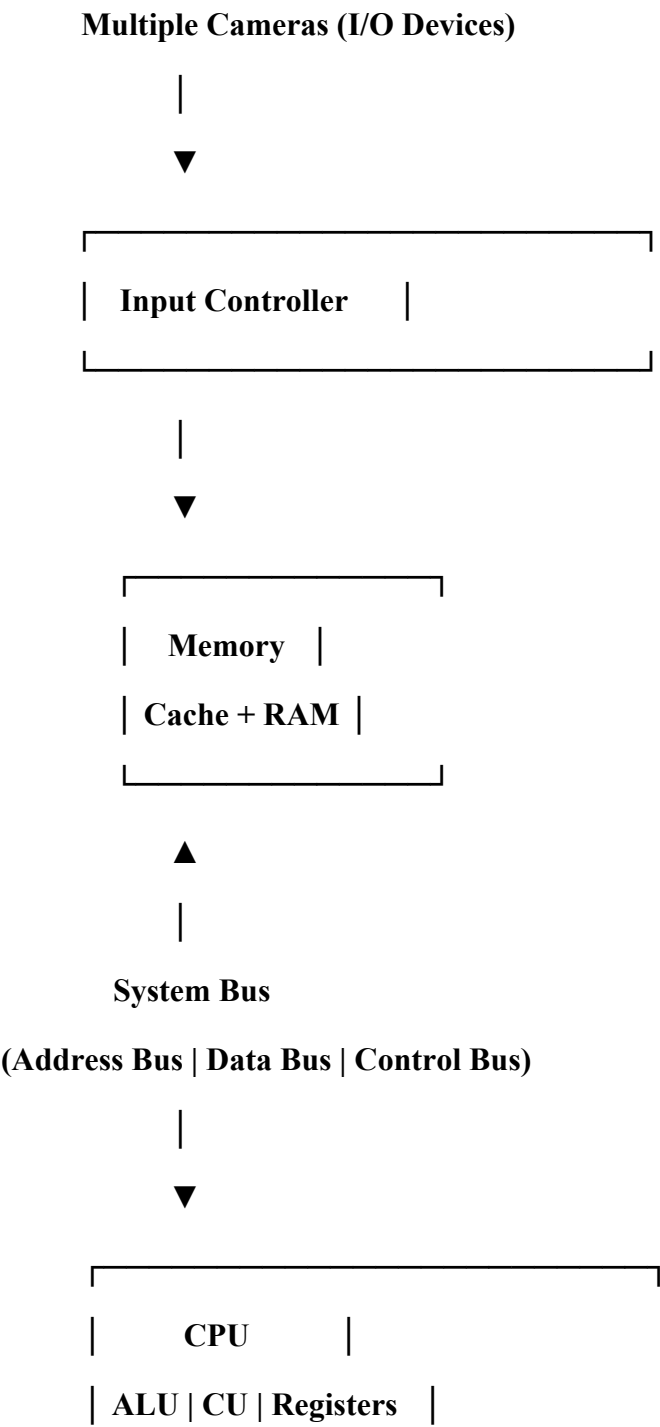
- Single-Bus Architecture – CPU, memory, and I/O devices share one common bus. Although it is simple and inexpensive, only one data transfer can occur at a time, creating bottlenecks during heavy workloads.
- Multi-Bus Architecture – Separate buses are used for processor-memory communication and I/O communication. This enables multiple data transfers simultaneously, improving throughput and reducing waiting time.
- Performance can be further enhanced by using cache memory, Direct Memory Access (DMA), pipelining, multicore processors, and parallel instruction execution. These architectural improvements reduce CPU idle time, minimize memory access delays, and increase overall system performance in real-time surveillance applications.

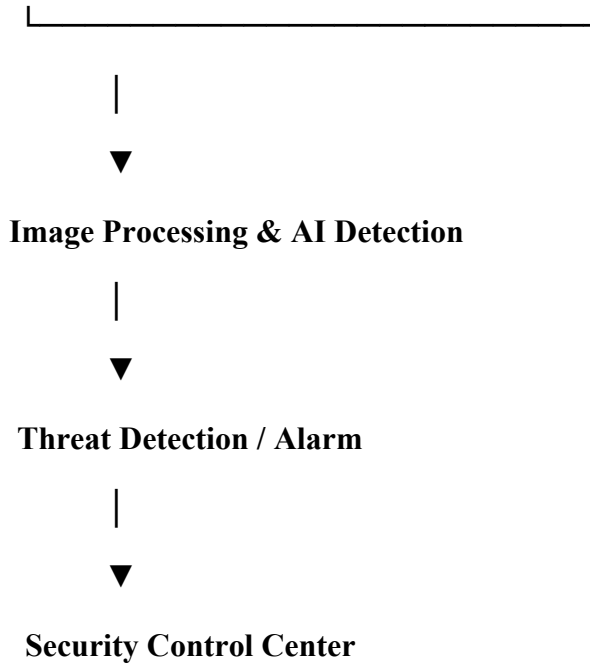
Problem Analysis:

The system experiences performance and reliability issues because hardware resources are not utilized efficiently. Bus contention, memory latency, incorrect addressing, improper instruction execution, and software design limitations can increase execution time, reduce throughput, and generate incorrect results. Identifying these bottlenecks helps in selecting suitable architectural improvements.

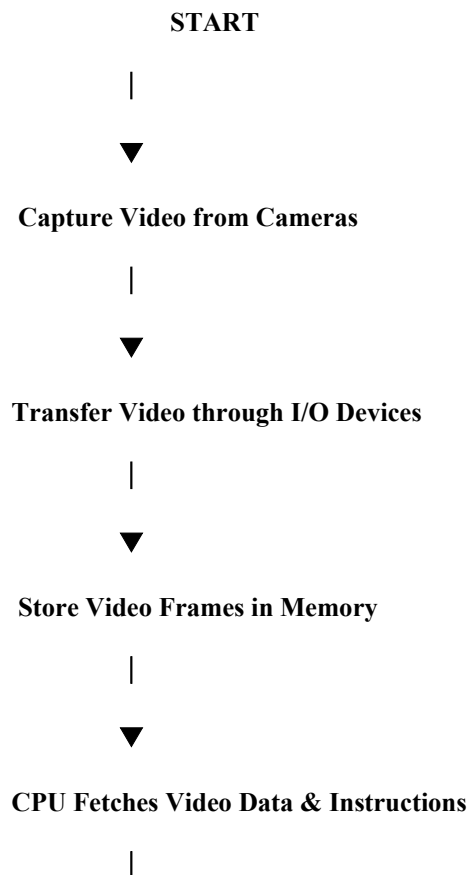
Block Diagram:

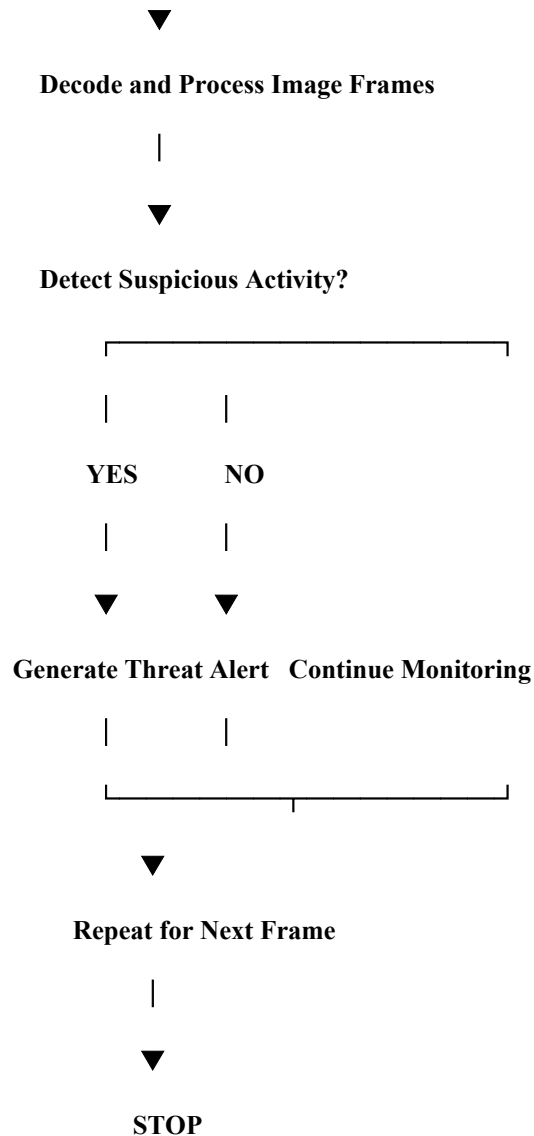
SMART CITY SURVEILLANCE SYSTEM





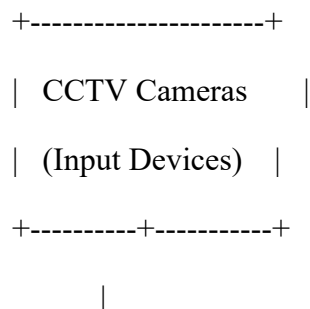
Flowchart:

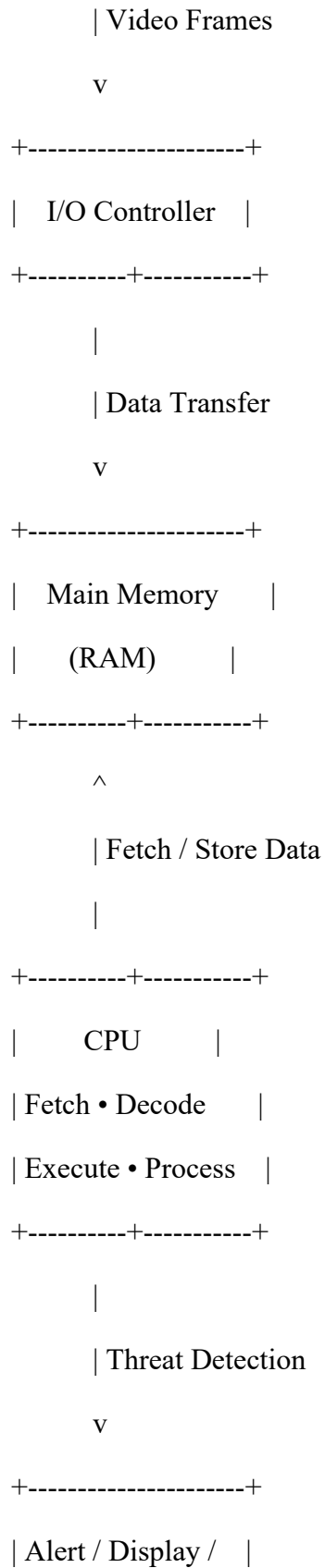




Example:

Suppose 100 cameras send Full HD video simultaneously. If the CPU uses a single bus, it must wait for memory access, causing delays. A multi-bus architecture allows multiple data transfers simultaneously, reducing lag.





| Police Notification |

+-----+

Pseudo Code:

START

Capture Video Frame

Store Frame in Memory

Fetch Frame to CPU

Process Image

IF Threat Detected THEN

Generate Alarm

ELSE

Continue Monitoring

ENDIF

Repeat

STOP

Advantages:

- Improves processing speed and efficiency.
 - Reduces execution time and latency.
 - Increases system reliability.
 - Supports better memory management.
 - Enhances multitasking and scalability.
 - Optimizes communication between CPU, memory, and I/O devices.
- Improved performance, reliability, scalability.

Disadvantages:

- Requires more complex hardware design.
 - multi-bus systems increase implementation cost.
 - Programming becomes more difficult with advanced addressing modes.
 - Incorrect memory or segment management may cause system errors.
 - Debugging low-level assembly programs requires significant expertise.
- Higher hardware cost and design complexity.

Conclusion

In conclusion, the discussed concepts demonstrate that system performance depends on efficient processor architecture, proper memory organization, optimized instruction execution, and correct software implementation. Selecting suitable bus structures, reducing memory latency, using appropriate addressing modes, and applying efficient programming techniques significantly improve overall performance. Proper testing, debugging, and hardware–software integration reduces errors and increase reliability. These principles are essential in modern embedded systems, industrial automation, communication systems, and high-performance computing applications.

Question 2 Real-Time Stock Trading System

Introduction

A real-time stock trading application is a high-performance computing system that executes millions of financial transactions every second. These applications are used by stock exchanges, brokerage firms, and financial institutions where even a delay of a few milliseconds can result in significant financial losses. The processor continuously executes instructions to process market data, verify user requests, calculate stock prices, and complete transactions. During periods of heavy market activity, the processor experiences increased workload due to frequent memory access and numerous branch instructions, resulting in a high Cycles Per Instruction (CPI). A high CPI means that the processor requires more clock cycles to execute each instruction, reducing the number of transactions processed per second. The efficiency of the Fetch–Decode–Execute cycle plays a crucial role in determining the overall execution time. Modern computer architectures use advanced techniques such as pipelining, branch prediction, cache memory, superscalar execution, and out-of-order execution to reduce CPI and improve processor performance. Understanding these architectural concepts enables engineers to develop faster and more efficient real-time trading systems capable of handling heavy transaction loads with minimal delay.

Theory:

The CPU execution cycle, also known as the Instruction Cycle, consists of three primary stages: Fetch, Decode, and Execute. During the Fetch stage, the processor retrieves instructions from the main memory into the instruction register. In the Decode stage, the control unit interprets the instruction and determines the required operation. During the Execute stage, the Arithmetic Logic Unit (ALU) performs arithmetic or logical operations, accesses memory if required, and stores the results.

The execution time of a processor is directly influenced by the Cycles Per Instruction (CPI), which represents the average number of clock cycles required to execute one instruction. A high CPI generally occurs due to:

- Frequent memory accesses
- Cache misses
- Branch instructions causing pipeline stalls
- Memory latency
- Data dependencies between instructions

Modern processors use several architectural techniques to reduce CPI:

- Instruction Pipelining allows multiple instructions to execute simultaneously.
- Branch Prediction reduces delays caused by conditional branches.
- Superscalar Architecture executes multiple instructions in one clock cycle.
- Out-of-Order Execution improves CPU utilization by executing independent instructions first.
- Cache Memory (L1, L2, L3) reduces memory access time.
- Prefetching loads instructions before they are required.
- Multicore Processing distributes workloads among multiple processor cores.

These techniques significantly improve throughput, reduce execution time, and enable real-time applications such as stock trading systems to process millions of transactions efficiently while maintaining low response time and high reliability.

Problem Analysis:

The system experiences performance and reliability issues because hardware resources are not utilized efficiently. Bus contention, memory latency, incorrect addressing, improper instruction execution, and software design limitations can increase execution time, reduce throughput, and generate incorrect results. Identifying these bottlenecks helps in selecting suitable architectural improvements.

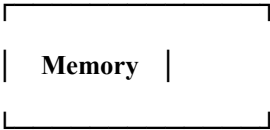
Block Diagram:

STOCK TRADING APPLICATION

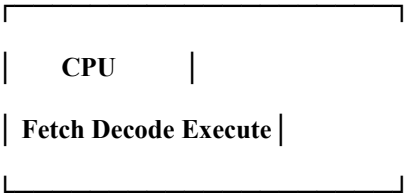
User Buy/Sell Request



Trading Application



System Bus



Branch Prediction Unit



Execute Trade





Update Database/Market

Flowchart:

START



Receive Buy/Sell Request



Store Request in Memory



Fetch Instruction from Memory



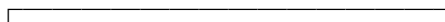
Decode Trading Instruction



Execute Buy/Sell Transaction

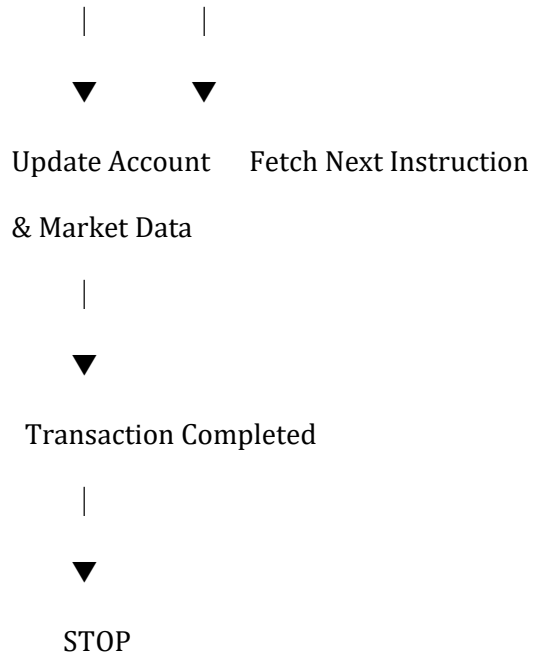


Branch Condition Satisfied?



YES

NO



Pseudo Code:

START

Read Order

Fetch Instruction

Decode Instruction

IF Buy THEN

Execute Buy

ELSE

Execute Sell

ENDIF

Update Database

STOP

Example:

If a processor executes 10 million instructions with a CPI of 4, execution takes longer than a processor with a CPI of 2.

Advantages:

- Improves processing speed and efficiency.
- Reduces execution time and latency.
- Increases system reliability.
- Supports better memory management.
- Enhances multitasking and scalability.
- Optimizes communication between CPU, memory, and I/O devices.

Improved performance, reliability, scalability.

Disadvantages:

- Requires more complex hardware design.
- Multi-bus systems increase implementation cost.
- Programming becomes more difficult with advanced addressing modes.
- Incorrect memory or segment management may cause system errors.
- Debugging low-level assembly programs requires significant expertise.

Higher hardware cost and design complexity.

Conclusion:

In conclusion, the discussed concepts demonstrate that system performance depends on efficient processor architecture, proper memory organization, optimized instruction execution, and correct software implementation. Selecting suitable bus structures, reducing memory latency, using appropriate addressing modes, and applying efficient programming techniques significantly improve overall performance. Proper testing, debugging, and hardware–software integration reduce errors and increase reliability. These principles are essential in modern embedded systems, industrial automation, communication systems, and high-performance computing applications.

Question 3

Introduction:

Embedded systems are specialized computing systems designed to perform dedicated functions efficiently and reliably. In industrial automation, embedded systems based on the 8085 microprocessor are widely used for monitoring and controlling physical processes such as temperature, pressure, speed, and humidity. In an industrial temperature control unit, temperature sensors continuously measure the surrounding temperature and send the data to the 8085 microprocessor. The processor analyzes the sensor values, compares them with predefined reference values, and controls output devices such as heaters, cooling fans, or alarms through actuators. The accuracy of this control process depends not only on the sensor hardware but also on the correct execution of the program. One of the most important factors affecting program correctness is the selection of the appropriate addressing mode. If an incorrect addressing mode is used, the processor may access the wrong memory location or register, resulting in incorrect temperature readings and improper control actions. Therefore, understanding the various addressing modes and effectively utilizing the 8085 instruction set is essential for developing a reliable and accurate industrial temperature control system.

Theory:

The 8085 microprocessor is an 8-bit microprocessor developed by Intel and is widely used in embedded systems due to its simple architecture, low cost, and efficient instruction set. It consists of an Arithmetic Logic Unit (ALU), accumulator, general-purpose registers, program counter, stack pointer, control unit, and timing circuitry. The processor executes instructions through the Fetch–Decode–Execute cycle, where each instruction is fetched from memory, decoded by the control unit, and executed by the ALU.

The 8085 instruction set supports several addressing modes, which determine how the processor accesses data during instruction execution. The five major addressing modes are:

1. Immediate Addressing Mode – The data is directly included in the instruction itself. Example: MVI A, 25H
2. Register Addressing Mode – Data is stored in processor registers. Example: MOV A, B
3. Direct Addressing Mode – The memory address of the data is specified explicitly. Example: LDA 2500H

4. Register Indirect Addressing Mode – The memory address is stored in a register pair such as HL. Example: MOV A, M
5. Implicit (Implied) Addressing Mode – The operand is implied by the instruction itself. Example: CMA

Selecting the appropriate addressing mode is critical in embedded applications because it determines how quickly and accurately data is accessed. In an industrial temperature controller, sensor readings are often stored in memory locations. If the program uses an incorrect addressing mode, the processor may retrieve data from an unintended memory location, leading to incorrect temperature calculations and faulty actuator responses. Therefore, programmers must carefully choose addressing modes and use suitable 8085 instructions to ensure accurate data processing, reliable system performance, and safe industrial operation.

Problem Analysis:

The system experiences performance and reliability issues because hardware resources are not utilized efficiently. Bus contention, memory latency, incorrect addressing, improper instruction execution, and software design limitations can increase execution time, reduce throughput, and generate incorrect results. Identifying these bottlenecks helps in selecting suitable architectural improvements.

Block Diagram:

TEMPERATURE CONTROL SYSTEM

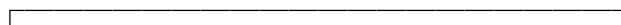
Temperature Sensor

|



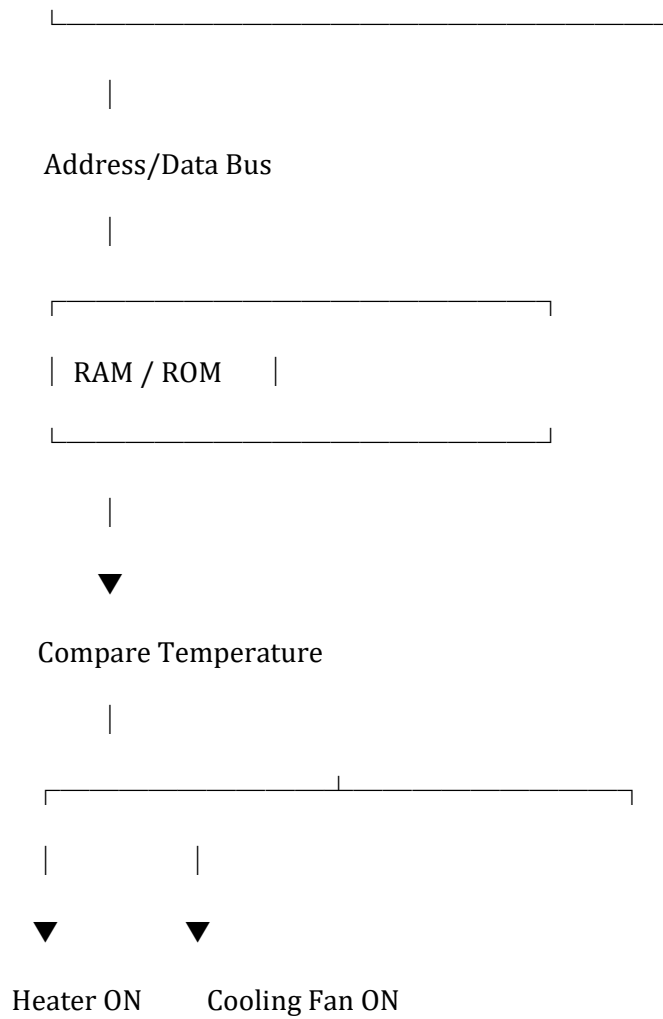
Analog to Digital Converter

|

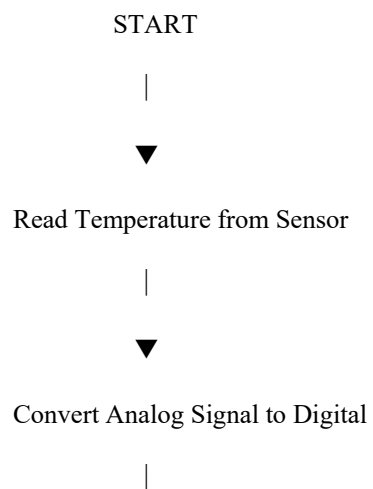


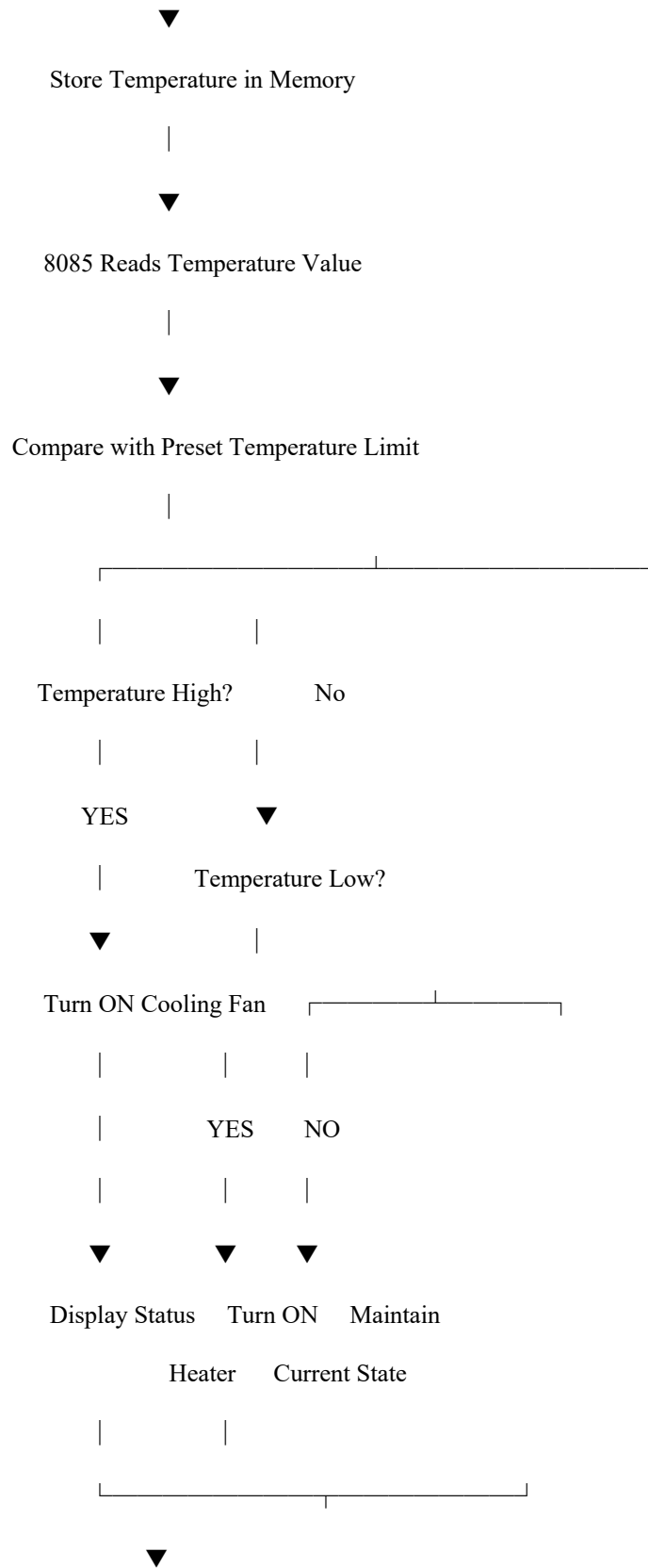
| 8085 Microprocessor |

| ALU CU Registers |



Flowchart:





Repeat Temperature Check

|



STOP

Pseudo Code:

LDA 2500H ; Read sensor value

MOV B,A

MVI A,32H ; Reference Temperature = 50°C

CMP B

JC COOLING

JZ NORMAL

JNC HEATING

COOLING:

MVI A,01H

OUT 01H

HLT

HEATING:

MVI A,02H

OUT 01H

HLT

NORMAL:

MVI A,00H

OUT 01H

HLT

EXAMPLE:

If the sensor measures 35°C but the processor reads data from the wrong memory location, it may display 80°C, causing incorrect control of the heating system.

Advantages:

- Improves processing speed and efficiency.
- Reduces execution time and latency.
- Increases system reliability.
- Supports better memory management.
- Enhances multitasking and scalability.
- Optimizes communication between CPU, memory, and I/O devices.

Improved performance, reliability, scalability.

Disadvantages:

- Requires more complex hardware design.
 - Multi-bus systems increase implementation cost.
 - Programming becomes more difficult with advanced addressing modes.
 - Incorrect memory or segment management may cause system errors.
 - Debugging low-level assembly programs requires significant expertise.
- Higher hardware cost and design complexity.

Conclusion:

In conclusion, the discussed concepts demonstrate that system performance depends on efficient processor architecture, proper memory organization, optimized instruction execution, and correct software implementation. Selecting suitable bus structures, reducing memory latency, using appropriate addressing modes, and applying efficient programming techniques significantly improve overall performance. Proper testing, debugging, and hardware–software integration reduce errors and increase reliability. These principles are essential in modern embedded systems, industrial automation, communication systems, and high-performance computing applications.

